PegaFlow: Tail-Predictable Host-Memory Pooling for Disaggregated LLM Serving

Advisor-authored storyline draft; implementation and experiments remain student-owned

September 2026

Abstract

Disaggregated LLM serving can enlarge its KV cache by pooling CPU memory on both prefill and decode nodes. The extra capacity is useful only if remote reads, refill, and migration do not turn shared RDMA into an uncontrolled tail latency path. We ask whether object placement and transfer budgets can be jointly controlled so that both host-memory pools extend useful KV residence while request p99 remains predictable. PegaFlow already contains host and SSD backing, a vLLM connector, cross-node transport, and exploratory serving traces. Those traces are not yet paper-admissible because historical code and runtime custody is incomplete; they establish neither an RDMA-tail pathology nor a benefit. We therefore formulate an object-lifecycle-aware placement and transfer mechanism, its strongest baselines, and a fail-closed evaluation contract. Cache TTL is treated as an outcome under a fixed total byte budget, not a configurable success metric. No new performance or NPU claim is made in this draft.

1 Problem and Representative Setting

Paged KV management makes cache capacity and fragmentation first-class serving concerns [1]. Systems such as Mooncake go further by using CPU memory, SSD, and NIC resources across disaggregated prefill and decode clusters [2]. This creates a stronger baseline than decode-local caching: both sides' memory can contribute to a distributed pool.

The representative setting is long-context P/D serving in which a decode node frequently consumes KV produced elsewhere. A remote hit can avoid recompute, but its critical path includes metadata lookup, memory registration, queueing, RDMA transfer, destination placement, and materialization. Concurrent refill or migration flows can share the same links with deadline-sensitive reads. Consequently, more retained KV may coexist with worse request p99.

Our research question is:

> Can a disaggregated KV store jointly choose host-memory placement, remote-read timing, and transfer budgets so that it uses CPU memory on both P and D nodes without making RDMA and request tail latency unpredictable?

The paper advances only if a matched real-runtime trace first shows the claimed counterexample: extra pooled capacity improves residency or hits while uncontrolled remote traffic causes a repeatable tail-latency penalty.

2 Gap and Scope

Capacity-oriented policies decide what to retain; transfer engines decide how to move bytes; serving schedulers decide where requests run. Treating these decisions independently assumes that a remote cache hit has a stable cost. That assumption can fail when large background movements and deadline-critical reads share registration resources, queues, links, or destination bandwidth.

The intended contribution is not another multi-NIC routing protocol and not a new KV correctness protocol. PegaFlow owns object placement, read admission, and transfer budgeting at the cache/runtime boundary. Multi-path selection, connection recovery, and generation-safe object identity remain required components or baselines, but are outside the paper's novelty claim.

3 Seven-Question Research Contract

Q	Frozen answer
1	**Problem.** Jointly exploit P- and D-node CPU memory while bounding the tail cost of remote KV reads and background movement.
2	**Importance.** A capacity win that violates TTFT or TPOT p99 is not a serving win; long-context requests make both saved compute and transferred bytes large.
3	**Gap.** Existing capacity, placement, and transport controls do not necessarily expose one request-level budget that accounts for queueing, registration, transfer, refill, and materialization. Mooncake is a strong end-to-end baseline, not evidence that the problem is already solved.
4	**Idea.** Give every transferable KV object a lifecycle receipt and a deadline-aware transfer value. Admit remote reads and background movement under separate bounded budgets; place replicas according to predicted saved compute, reuse distance, bytes, and contention.
5	**Feasibility.** PegaFlow already exposes host backing, topology, leases, prefetch, read cache, RDMA, connector, and metrics seams. The first scope is one P/D deployment and one transfer path.
6	**Evaluation.** Freeze total CPU memory, workload, topology, runtime, and request order; compare local-only, static pooled memory, NUMA-aware placement, transfer shaping, Mooncake-compatible policy, and the joint design.
7	**Takeaway.** Distributed cache capacity must be scheduled together with transfer debt; the useful region is predictable from saved compute, reuse distance, object bytes, and deadline slack.

4 Mechanism Hypothesis

For object i, the controller estimates

$$V_i = C_i^{\text{saved}} - (L_i^{\text{lookup}} + L_i^{\text{queue}} + L_i^{\text{xfer}} + L_i^{\text{materialize}}),$$

along with bytes, expected reuse, deadline slack, source/destination pressure, and placement lifetime. It chooses among local retain, remote retain, replicate, defer, evict, or recompute. Deadline-critical reads receive a bounded foreground budget; refill, replication, and migration consume a separately capped background budget. A decision is revoked when identity, generation, topology, or measured queue state differs from its admission receipt.

This is not merely rate limiting. Placement changes future traffic, while traffic debt changes whether a remote copy is valuable. The hypothesis predicts wins when reuse is high, saved prefill is large, and remote queues are controllable. It predicts losses for short prefixes, one-shot objects, saturated links, inaccurate reuse estimates, or materialization-dominated paths.

5 Current Evidence Ledger

The repository preserves real-online trace families and a portable trace-audit harness. Issue 18 records that most historical referenced commits were not reachable from durable refs at review time. Those artifacts remain exploratory: they may select variables and failure cases, but no numeric result enters this paper. Simulation and host probes qualify interfaces only.

Table 2: Evidence boundary before the new matched run.

Artifact	Class	Admissible use
Historical serving traces	exploratory real-online	variable selection; no number
Trace preregistration	contract	matched-arm definition
Host/simulation tests	qualification	semantics only
P/D RDMA tail counterexample	absent	no pathology claim
Joint policy result	absent	no benefit claim

6 Matched Evaluation Contract

Trace schema. Each object records producer, consumer, model/request/generation identity, bytes, source and destination tier, creation, lookup, queue, transfer, materialization, hit/miss, reuse distance, deadline, cancellation, and cleanup. NIC/link and NUMA identities must be tied to the request receipt.

Arms. Under identical model, requests, P/D counts, total CPU-memory bytes, topology, and network conditions, compare: decode-local only; static use of both host pools; NUMA-aware static placement; bounded-concurrency or priority shaping; a deployable Mooncake-style baseline; and joint placement plus transfer budget.

Metrics. Report at least ten independent starts and full distributions for RDMA queue delay, flow concurrency, bytes, useful bandwidth, cache hit, effective residence time, TTFT, TPOT, throughput, recompute, recovery, and memory overhead. All arms require completion, output, object identity, generation, cleanup, and capacity conservation oracles.

Success gate. The primary gate is a preregistered reduction in RDMA and request p99 relative to the strongest deployable baseline at unchanged total CPU memory and non-inferior cache hit. Baseline repeats must freeze the numerical margin before treatment data are inspected. A 50% increase in effective cache residence is reported only as a secondary capacity benefit and cannot compensate for worse tail latency.

7 Stopping Rules and Claim Discipline

Stop the joint mechanism if matched real traces do not show an uncontrolled traffic tail, if static placement or simple shaping reaches the oracle, or if remote reads cannot beat recompute in the tested regime. Close only that mechanism and retain the correctness carrier.

The present Ascend code establishes a porting/adaptation seam, not an architecture-native contribution. An Ascend claim requires documented runtime or memory behavior, a matched counterexample on 910B2, a native response, and a predictive boundary. Host replay does not establish RDMA or serving benefit.

8 Conclusion

PegaFlow's paper question is not whether unused host memory can hold more KV. It is whether distributed capacity can be made useful without importing unbounded transfer debt into the request critical path. The next decisive artifact is therefore a custody-complete P/D trace that jointly exposes residency and tail cost, followed by the frozen matched comparison above.

References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023.

[2] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, and Weimin Zheng. Mooncake: Trading more storage for less computation—a kvcache-centric architecture for serving llm chatbot. In *23rd USENIX Conference on File and Storage Technologies*, pages 155–170, 2025.